



GIOCO DELL'OCA

Fase di Elaborazione – Iterazione 2

CALOGERO FALCONE & FRANCESCO CATANIA

Sommario

1.0	Introduzione	1
2.0	Caso d'uso UC2: Partita singleplayer.....	1
3.0	Analisi Orientata agli Oggetti	5
3.1	Modello di Dominio	5
3.2	Diagramma di Sequenza di Sistema	7
3.3	Contratti delle operazioni	8
4.0	Progettazione Orientata agli Oggetti	10
4.1	Diagramma di Sequenza	10
4.2	Diagramma delle Classi	12
4.3	Caso d'Uso di Avviamento	12

1.0 Introduzione

Durante la seconda iterazione verranno analizzati i seguenti requisiti:

- Implementazione dello scenario principale di successo del caso d'uso *UC2: Partita singleplayer*.
- Aggiornamento di alcune specifiche introdotte con il caso d'uso *UC1: Configurazione e avvio partita singleplayer* per gestire le esigenze di sviluppo per questa iterazione.

2.0 Caso d'uso UC2: Partita singleplayer

Di seguito viene riportato nel dettaglio il caso d'uso *UC2: Partita singleplayer*. Questo caso d'uso permette al giocatore di entrare in una partita, precedentemente configurata tramite l'*UC1*, e giocare contro il computer. Questo caso d'uso non presenta sostanziali modifiche rispetto al caso d'uso presentato in fase di ideazione.

Caso d'uso

	Partita singleplayer
Nome del caso d'uso	UC2: Partita singleplayer
Portata	Applicazione Gioco dell'Oca
Livello	Obiettivo Utente
Attore primario	Giocatore

Parti interessate e interessi	Giocatore → Vuole giocare una partita singleplayer al Gioco dell'Oca
Pre-condizioni	Il giocatore deve aver configurato e avviato una partita singleplayer
Garanzia di successo	Il sistema conclude la partita precedentemente avviata
Scenario principale di successo	1. Il sistema imposta l'utente come primo giocatore di turno. 2. Il sistema controlla lo stato del giocatore di turno. 3. L'utente inizia il proprio turno cliccando sul pulsante "Lancia dadi". 4. Il sistema genera N numeri casuali da 1 a 6. 5. La pedina del giocatore di turno si muove in avanti della somma dei numeri che scaturiscono dal lancio dei dadi. 6. In base alla casella del tabellone sulla quale finisce la pedina del giocatore di turno, viene applicato l'effetto appropriato. 7. Il sistema salva il numero della casella sulla quale finisce la pedina del giocatore di turno. 8. Il sistema controlla che la casella sulla quale è finita la pedina del giocatore di turno sia quella finale. 9. Il sistema conclude la partita, indicando come vincitore il giocatore la cui pedina è finita sulla casella finale. 10. L'utente clicca sul pulsante "Torna al menu principale". 11. Il sistema torna alla pagina del menu principale.
Estensioni	1a. In qualsiasi momento, l'utente può cliccare sul pulsante "Torna al menu principale". 1. Il sistema chiede conferma della scelta, avvisando l'utente che perderà i progressi relativi alla partita in corso. 2. L'utente decide se abbandonare la partita o meno. 2a. L'utente clicca su "Sì". 1. Il sistema apre la pagina del menu principale, perdendo traccia dei progressi relativi alla partita in corso. 2b. L'utente clicca su "No". 1. Il sistema permette di continuare normalmente la partita.

1b. In qualsiasi momento, il sistema subisce un arresto improvviso.

1. L'utente ripristina il sistema.
2. Il sistema ripristina lo stato della partita.

2a. Il giocatore di turno è nello stato "Bloccato".

1. Il sistema controlla chi è il giocatore di turno.

1a. Il giocatore di turno è l'utente.

1. Il sistema imposta come nuovo giocatore di turno il computer e continua con il passo 4.

1b. Il giocatore di turno è il computer.

1. Il sistema imposta come nuovo giocatore di turno l'utente e continua con il passo 3.

2b. Il giocatore NON è nello stato "Bloccato".

1. Il sistema controlla chi è il giocatore di turno.

1a. Il giocatore di turno è l'utente.

1. Il sistema continua con il passo 3.

1b. Il giocatore di turno è il computer.

1. Il sistema continua con il passo 4.

6a. La casella sulla quale è finito il giocatore di turno è una casella normale.

1. La casella sulla quale finisce la pedina viene impostata alla casella attuale e il sistema continua con il passo 7.

6b. La casella sulla quale è finito il giocatore di turno è una casella speciale di tipo "Oca".

1. La nuova casella sulla quale finisce la pedina del giocatore di turno sarà calcolata come "Casella attuale" + "Somma dei numeri ottenuti dal lancio dei dadi".

6c. La casella sulla quale è finito il giocatore di turno è una casella speciale di tipo "Ponte".

1. La nuova casella sulla quale finisce la pedina del giocatore di turno sarà calcolata come "Casella attuale" * 2.

6d. La casella sulla quale è finito il giocatore di turno è una casella speciale di tipo "Locanda".

1. Il sistema imposta lo stato del giocatore di turno a "Bloccato" per il turno successivo.

Requisiti speciali

Elenco delle varianti tecnologiche e dei dati

Frequenza di ripetizioni

Varie

6e. La casella sulla quale è finito il giocatore di turno è una casella speciale di tipo "Prigione".

1. Il sistema controlla se sulla stessa casella è presente un altro giocatore.

1a. Sulla stessa casella NON è presente un altro giocatore.

1. Il sistema imposta lo stato del giocatore di turno a "Bloccato" indefinitamente.

1b. Sulla stessa casella è presente un altro giocatore.

1. Il sistema imposta lo stato giocatore che è finito sulla casella a "Bloccato" indefinitamente e imposta lo stato del giocatore che era già presente sulla casella a "Sbloccato".

6f. La casella sulla quale è finito il giocatore di turno è una casella speciale di tipo "Labirinto".

1. La nuova casella sulla quale finisce la pedina del giocatore di turno sarà calcolata come "Casella attuale" - 3.

6g. La casella sulla quale è finito il giocatore di turno è una casella speciale di tipo "Scheletro".

1. La nuova casella sulla quale finisce la pedina del giocatore di turno viene reimpostata alla casella di inizio.

8a. La casella sulla quale è finito il giocatore di turno NON è quella finale.

1. Il sistema controlla chi è il giocatore di turno.

1a. Il giocatore di turno è l'utente.

2. Il sistema imposta come nuovo giocatore di turno il computer e ritorna al passo 4.

1b. Il giocatore di turno è il computer.

2. Il sistema imposta come nuovo giocatore di turno l'utente e ritorna al passo 3.

8b. La casella sulla quale è finito il giocatore di turno è quella finale.

1. Il sistema continua con il passo 9.

Dipende dal numero di partite singleplayer che il giocatore vuole avviare

3.0 Analisi Orientata agli Oggetti

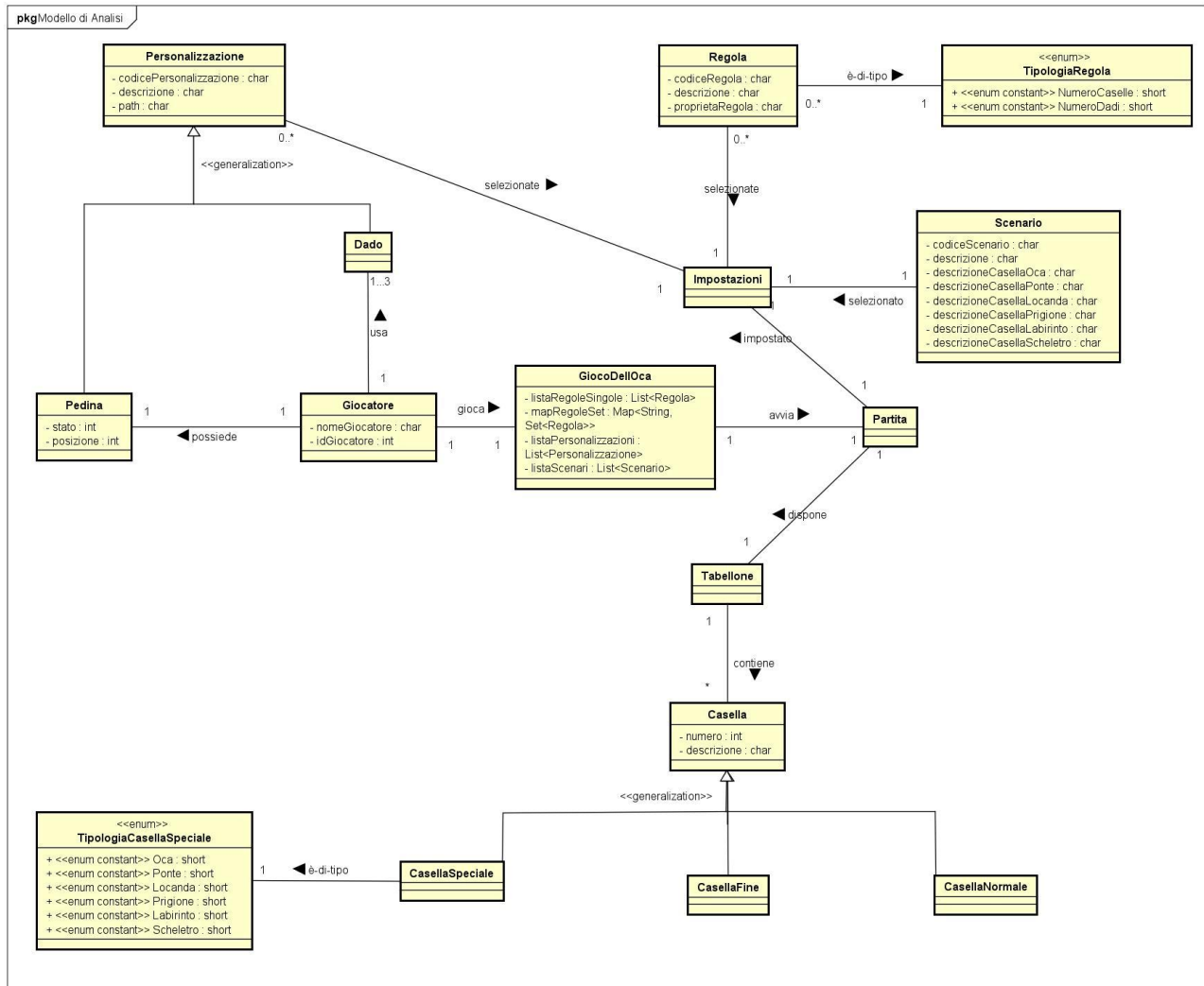
Anche in questo caso verranno impiegati gli stessi strumenti dell'*Iterazione 1* per fornire la descrizione del dominio, ovvero: il *Modello di Dominio*, i *Diagrammi di Sequenza di Sistema* (SSD) e i *Contratti delle Operazioni*.

3.1 Modello di Dominio

Bisogna definire il Modello di Dominio, ovvero un elaborato grafico che permette di identificare i concetti, gli attributi e le associazioni più importanti. Dall'analisi del primo caso d'uso preso in considerazione, valutando lo scenario principale di successo, è stato possibile identificare le seguenti classi concettuali:

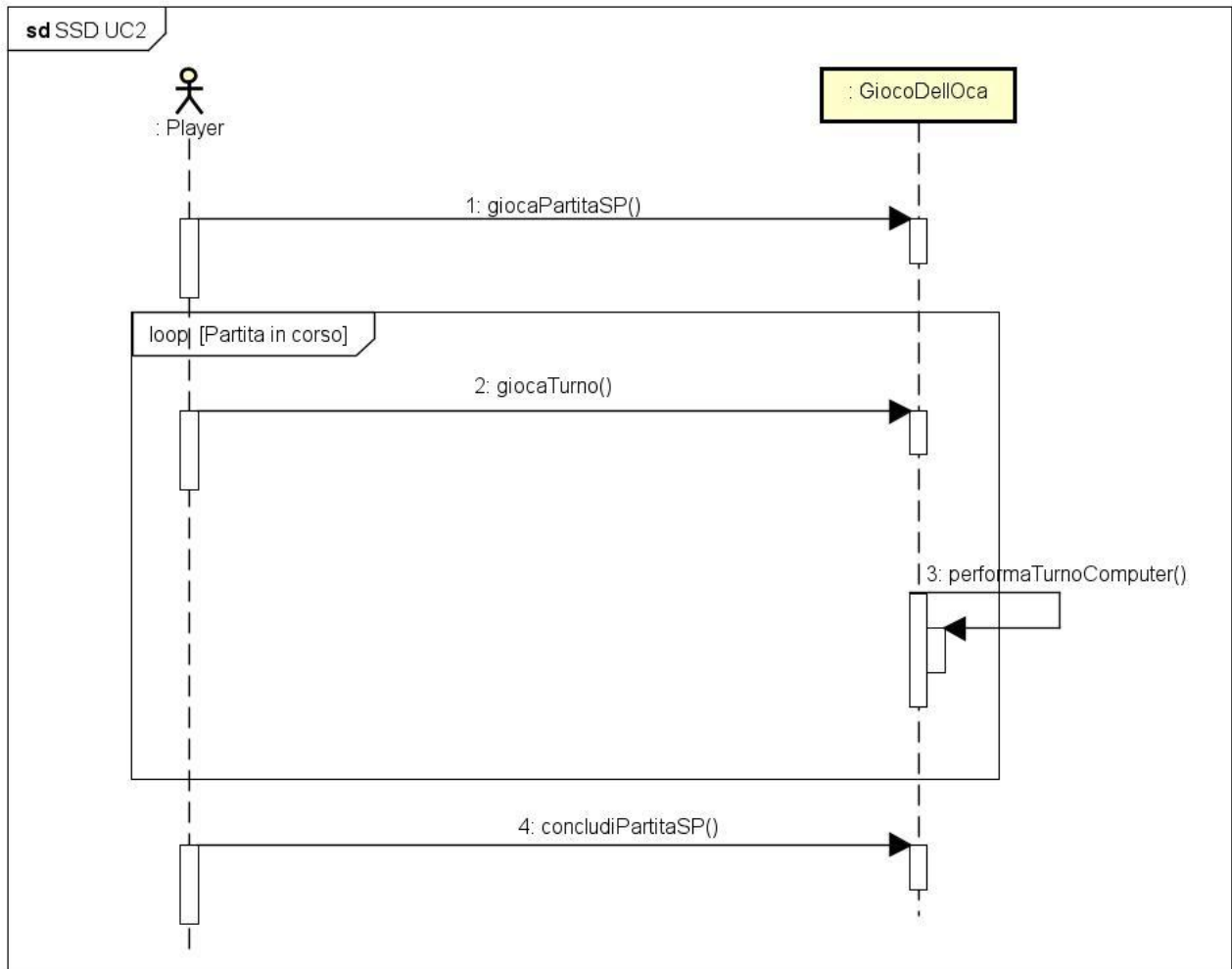
- **Giocatore**: rappresenta l'attore primario, che interagisce con il sistema per eseguire le operazioni, viene definito nell'Iterazione 1;
- **GiocoDellOca**: rappresenta il sistema che permette la gestione della partita, viene definito nell'Iterazione 1;
- **Partita**: entità che rappresenta la sessione di gioco (in questo caso singolo giocatore) che dispone di impostazioni iniziali per il suo funzionamento, viene definito nell'Iterazione 1;
- **Impostazioni**: entità che gestisce regole, scenari e personalizzazioni che verranno usati in partita, viene definito nell'Iterazione 1;
- **Scenario**: lo scenario definisce la narrazione che sottintende ogni partita: inciderà dunque sulla descrizione di ogni casella. Viene scelto uno scenario all'inizio di ogni partita, viene definito nell'Iterazione 1;
- **Regola**: le regole rappresentano la modalità di gioco (rapida, normale o lunga) e il numero di dadi che verranno utilizzati dai giocatori nella partita, viene definito nell'Iterazione 1. Le diverse tipologie che una regola può assumere sono stabilite da **TipologiaRegola**.
- **Personalizzazione**: rappresentano le proprietà di alcuni elementi di gioco sotto il diretto controllo del giocatore modificabili a piacimento:
 - **Pedina**: rappresenta il Giocatore sulle Caselle del Tabellone. Viene impostato dal Giocatore tramite Impostazioni all'inizio della Partita secondo quanto definito nella Iterazione 1;
 - **Dado**: oggetto che serve per il movimento della Pedina lungo il Tabellone. Viene impostato dal Giocatore tramite Impostazioni all'inizio della Partita secondo quanto definito nella Iterazione 1;
- **Tabellone**: si tratta del campo di gioco messo a disposizione dal controller GiocoDellOca tramite Partita. L'interazione con l'oggetto Tabellone non è diretta, ma passa attraverso le Caselle che lo costituiscono;
- **Casella**: rappresentano le varie posizioni che il Giocatore tramite la Pedina può assumere. La Casella può essere di tre tipi: **CasellaNormale**, **CasellaSpeciale**, **CasellaFine**. Se il Giocatore finisce su una **CasellaSpeciale** viene attivato un modificatore che provoca sulla Pedina un avanzamento, un indietreggiamento o un blocco (intervento sullo stato), secondo specifiche regole definite in base alla **TipologiaCasellaSpeciale**.

Tenendo conto di associazione e attributi, si costruisce il seguente Modello di Dominio:



3.2 Diagramma di Sequenza di Sistema

Procedendo con la **OOA**, si passa ora alla creazione del Diagramma di Sequenza di Sistema (SSD) per poter descrivere il corso degli eventi di I/O per lo scenario principale di successo del caso d'uso scelto, ovvero l'*UC2: Partita singleplayer*. Il diagramma sarà il seguente:



3.3 Contratti delle operazioni

Attraverso l'utilizzo dei Contratti delle Operazioni, verranno descritte le principali operazioni di sistema responsabili della gestione degli eventi rilevati e analizzati.

Contratto C01 <i>giocaPartitaSP</i>	
Operazioni	<code>giocaPartitaSP();</code>
Riferimenti	Caso d'uso UC2: Partita singleplayer;
Pre-condizioni	È stata creata una istanza <code>partitaCorrente</code> ed è stata avviata la partita secondo il Caso d'uso UC1.
Post-condizioni	- <i>partitaCorrente</i> crea un'istanza di <i>Tabellone</i> e n istanze di <i>Casella</i>, personalizzando il campo da gioco in base alle regole e allo scenario precedentemente selezionati; - <i>partitaCorrente</i> associa al giocatore i dadi e la pedina, in base alle scelte effettuate in fase di configurazione.

Contratto C02 <i>giocaTurno</i>	
Operazioni	<code>giocaTurno();</code>
Riferimenti	Caso d'uso UC2: Partita singleplayer;
Pre-condizioni	È stata creata una istanza <code>partitaCorrente</code> ed è stata avviata la partita secondo il Caso d'uso UC1.
Post-condizioni	- Se la pedina associata al giocatore ha uno stato diverso da "Bloccato", il giocatore lancia i dadi e il valore ottenuto dal lancio viene utilizzato per determinare il numero della nuova Casella che verrà associata alla pedina; - Qualora la Casella sia di tipo <i>CasellaSpeciale</i>, viene effettuato un controllo sulla <i>TipologiaCasellaSpeciale</i>: in base a tale proprietà, verranno determinati sia il numero della nuova Casella da associare alla Pedina, sia un eventuale cambiamento di stato della stessa. - Se la Casella è di tipo <i>CasellaFine</i> si procede con l'esecuzione del metodo <code>concludiPartita()</code>.

Contratto C03 *performaTurnoComputer*

Operazioni	<code>performaTurnoComputer();</code>
Riferimenti	Caso d'uso UC2: Partita singleplayer;
Pre-condizioni	È stata creata una istanza <code>partitaCorrente</code> ed è stata avviata la partita secondo il Caso d'uso UC1. Il giocatore umano ha già effettuato i passaggi del <code>giocaTurno</code> per il turno corrente
Post-condizioni	- Se la pedina associata al bot ha uno stato diverso da "Bloccato", il giocatore lancia i dadi e il valore ottenuto dal lancio viene utilizzato per determinare il numero della nuova Casella che verrà associata alla pedina; - Qualora la Casella sia di tipo <code>CasellaSpeciale</code>, viene effettuato un controllo sulla <code>TipologiaCasellaSpeciale</code>: in base a tale proprietà, verranno determinati sia il numero della nuova Casella da associare alla Pedina, sia un eventuale cambiamento di stato della stessa. - Se la Casella è di tipo <code>CasellaFine</code> si procede con l'esecuzione del metodo <code>concludiPartita()</code>.

Contratto C04 *concludiPartita*

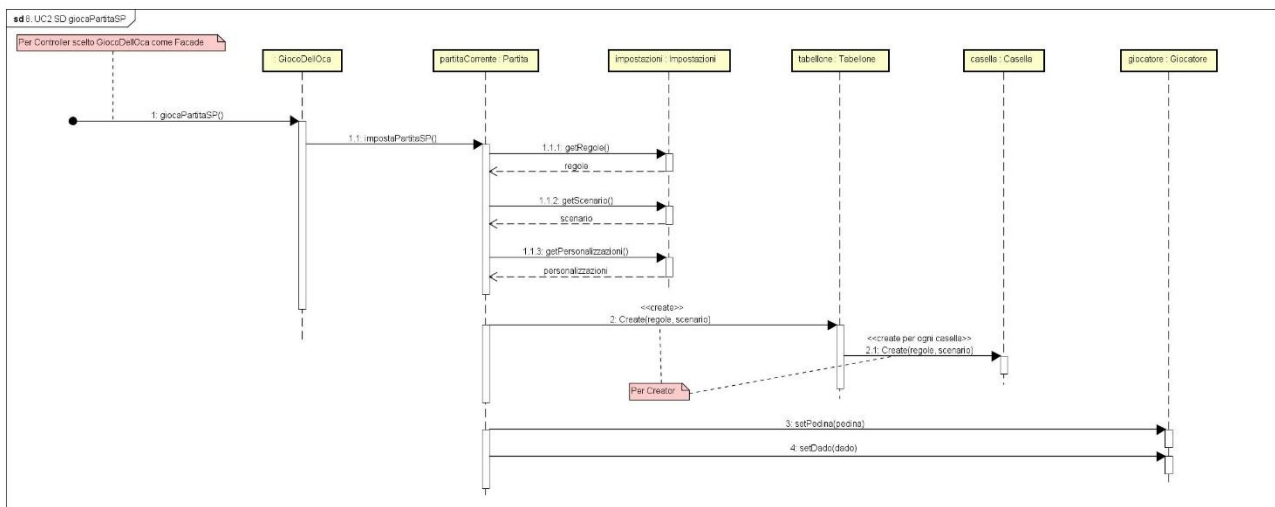
Operazioni	<code>concludiPartita();</code>
Riferimenti	Caso d'uso UC2: Partita singleplayer;
Pre-condizioni	È stata creata una istanza <code>partitaCorrente</code> ed è stata avviata la partita secondo il Caso d'uso UC1. Il giocatore umano e il giocatore bot hanno già effettuato i passaggi del <code>giocaTurno</code> per almeno un turno
Post-condizioni	- Viene effettuato un controllo tramite la posizione della Pedina se la Casella su cui si trova è di tipo <code>CasellaFine</code>; - Se la Casella è di tipo <code>CasellaFine</code>, il gioco finisce e compare a schermo il nome del vincitore e la possibilità di accedere al menu iniziale (secondo quanto scritto nel caso d'uso UC1).

4.0 Progettazione Orientata agli Oggetti

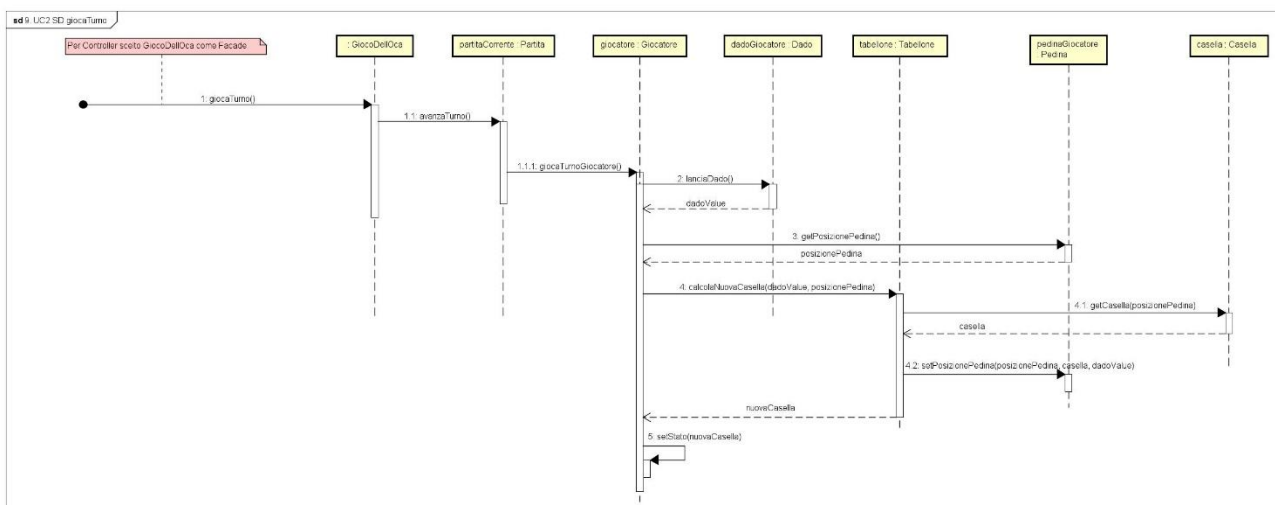
Di seguito, in relazione al Modello di Progetto, vengono presentati i diagrammi più significativi riguardanti il caso d'uso *UC2: Partita singleplayer*.

4.1 Diagramma di Sequenza

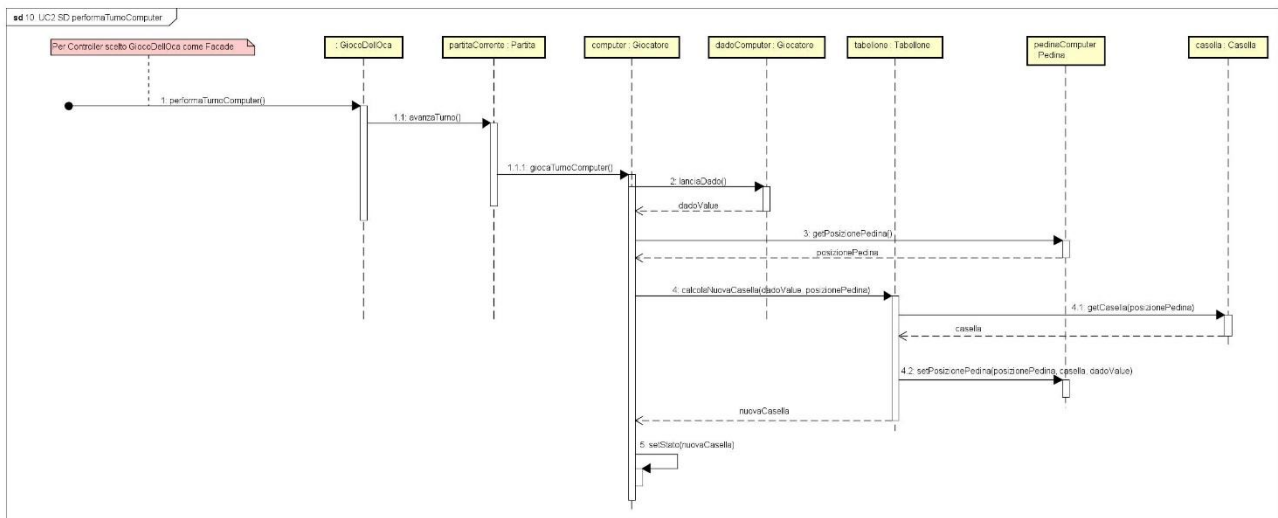
SD – giocaPartitaSP



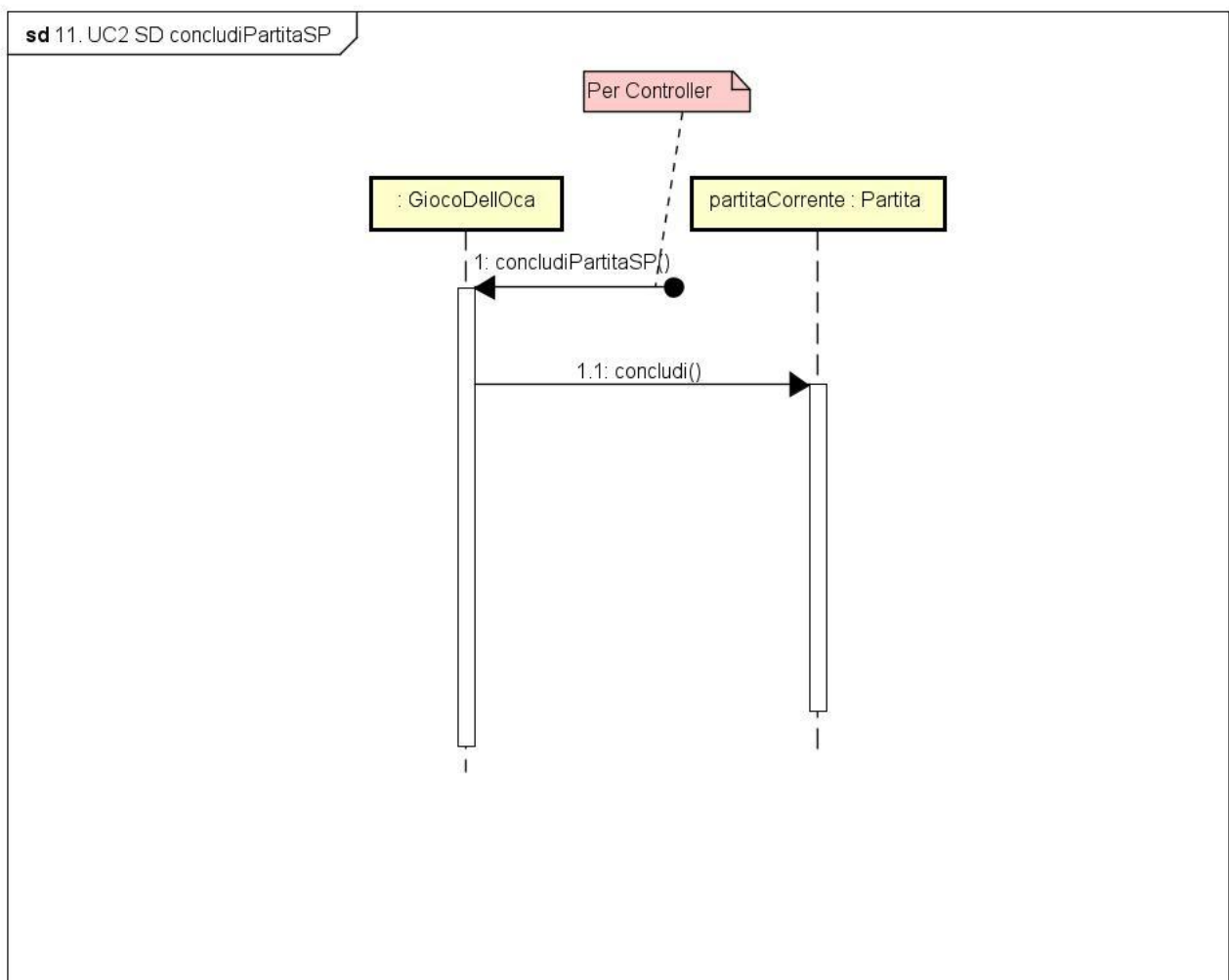
SD – giocaTurno



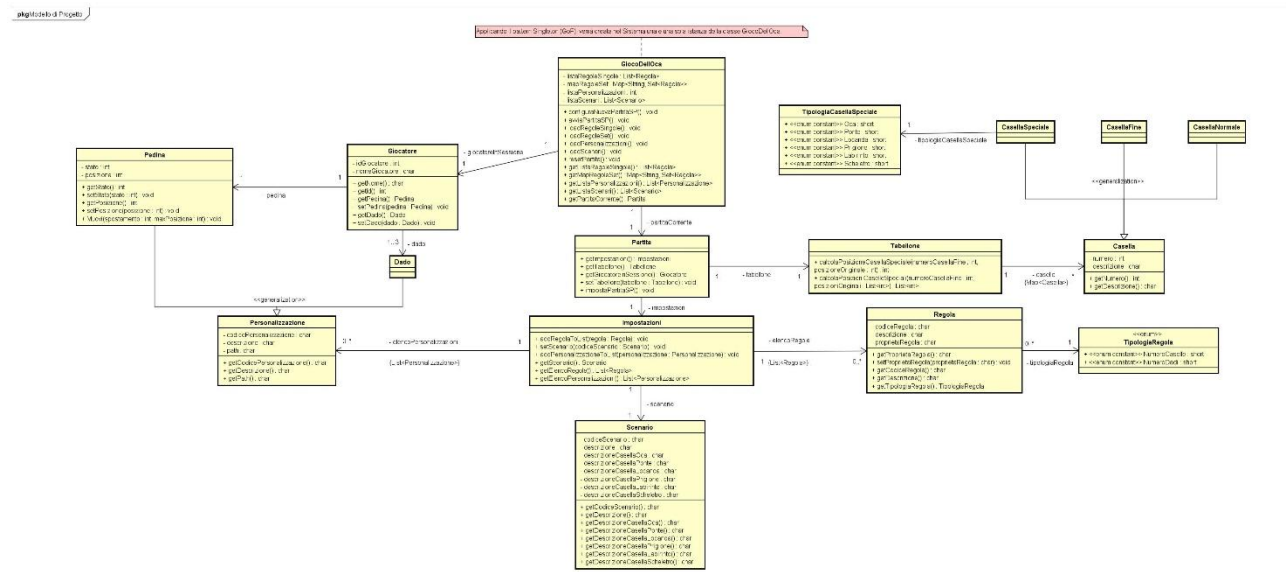
SD – performaTurnoComputer



SD – concludiPartitaSP



4.2 Diagramma delle Classi



4.3 Caso d'Uso di Avviamento

Il Caso d'Uso di Avviamento non presenta modifiche rispetto a quello presentato durante l'Iterazione 1.

5.0 Implementazione

Per la seconda iterazione sono state seguite le stesse scelte progettuali e implementative della prima iterazione, ovvero:

- Si è scelto di non utilizzare un database per memorizzare i dati in maniera permanente, ma di mantenerli in memoria principale e caricarli in fase di avviamento.;
- È stata implementata una semplice interfaccia tramite Java Swing.

Per questa iterazione, gran parte della logica di gestione della partita è stata mantenuta nella classe GiocoDellaOcaGUI e sarà soggetta a *refactoring* nelle iterazioni successive, in modo da utilizzare la suddetta classe solo per gestire le componenti Java Swing.

6.0 Testing

I test sono stati eseguiti sulle seguenti classi:

- **Pedina:**
 - Muovi: ci si accerta che, data una posizione iniziale, la pedina si muova fino alla posizione finale corretta, tenendo conto anche del caso di “rimbalzo” dovuto alla fine delle caselle del tabellone.
- **Tabellone:**
 - Costruttore: ci si accerta che il tabellone venga creato correttamente.